

# Technical Brief:

## Fee Proposal Generation System

### 1 Overview

The Fee Proposal Generation System (FP-GEN) is a software tool designed to automatically produce comprehensive fee proposal documents in response to client Requests for Proposals (RFPs). These fee proposals follow a standard structure – including sections such as Cover Letter, Project Understanding, Methodology, Scope and Deliverables, Schedule, Team Structure, Financial Proposal, and Relevant Experience – which are all assembled into a formatted document. The system leverages automation (including Large Language Model integration) to parse the RFP and populate proposal content, uses database-driven data (e.g. staff rates, past project profiles), and employs template-based document generation to ensure consistency and efficiency. This brief details the input requirements, output content, calculation algorithms, user interface design, and automation logic, providing a guide for development and maintenance teams.

### 2 Input Data Requirements

To generate a tailored proposal, the system must gather various input data fields from either the RFP document or internal databases. Each field's source, format, and necessity are defined below:

#### 2.1 **Project Information:**

- 2.1.1. **Project Name:** Extracted from the RFP (e.g. project title or RFP cover) or entered by the user. Text string, mandatory (used throughout the proposal document, especially in the Cover Letter and headers).
- 2.1.2. **Client Name:** Obtained from the RFP (client organization issuing the RFP) or user input. Text string, mandatory (used in the Cover Letter salutation and elsewhere).
- 2.1.3. **Project Location:** From RFP context or user input – city, country, or region of the project. Text string, optional but recommended (appears in Project Understanding and Relevant Experience matching).
- 2.1.4. **Project Type/Category:** Identified from the RFP (e.g. “Master Plan”, “Office Tower Design”) or selected by user. Text, optional (can trigger selection of methodology templates and relevant experience examples).

2.1.5. **Site Area/Size:** If provided in RFP (e.g. “50 hectares”), numeric with units, optional (included in Project Understanding to demonstrate scale).

2.1.6. **Project Duration or Timeline:** If RFP specifies a required completion time or schedule (e.g. “complete in 11 weeks”), numeric (weeks or months) or date range, optional (used in Schedule section and to plan phases).

## 2.2 Scope & Deliverables:

2.2.1 **Scope of Work Summary:** A descriptive summary of what the project entails and what the client expects. Extracted by parsing the RFP’s scope description and requirements. Text (multi-sentence), mandatory (feeds into the Project Understanding section and informs methodology).

2.2.2 **Deliverables List:** Specific deliverables the client expects (e.g. reports, drawings, models). Extracted from the RFP (often listed in a scope section) or entered by the user if not explicitly stated. Format: a list of deliverable items (bulleted), mandatory (populates the Scope and Deliverables section).

2.2.3 **Project Phases/Stages:** The phases of work as defined by the RFP or as standard for the project type (e.g. Concept, Detailed Design, etc.). Could include phase names and descriptions. If the RFP outlines a phased approach, those are parsed; otherwise user can define phases. Format: list of phase names (with optional brief descriptions), mandatory for fee breakdown (even if a single phase).

2.2.4 **Milestones/Workshops:** Key meetings or milestones (such as workshops, presentations, submission dates) expected by the client. Extracted from RFP schedule requirements or set by user based on methodology. Format: list of events with timing, optional (used to enrich the Schedule and Methodology sections).

## 2.3 Team & Resources:

2.3.1 **Required Disciplines/Expertise:** Based on RFP requirements or project type (e.g. Urban Planning, Architecture, Infrastructure, Landscape). Parsed from RFP if it specifies required expertise, otherwise inferred. Format: list of disciplines, optional (guides team composition and mentioned in Cover Letter and Team Structure).

2.3.2 **Proposed Team Members/Structure:** The internal team roles proposed for the project. This includes key personnel (e.g. Project Director, Project Manager, discipline leads) and any partner or specialty consultants. Sourced from an internal database of staff or entered by the proposal manager. Format: for each team role, name/title/department; optional but typically provided (populates the

Team Structure section and sometimes mentioned in Cover Letter).

- 2.3.3 Specialty Consultants:** Any external specialist firms or consultants to be engaged (e.g. local partners, environmental consultants). Often identified in the RFP or decided by the firm. Format: name of consultant and role (e.g. “Local planning partner”), optional (if none required). These may be included in the Team Structure and noted in the Financial Proposal (usually as separate cost items or exclusions).

## **2.4 Financial Inputs:**

- 2.4.1 Standard Rates for Personnel:** Retrieved from the company’s rate database for all relevant staff roles/ranks. Each role has an hourly or daily rate (and possibly an overhead component) defined. For example, the database might list standard hourly rates for ranks like *Planner P-ST3* or *Architect A-PR4*, including base staff cost and indirect cost. *Source:* internal database; *Format:* numeric (e.g. in JPY/hour or per day), required for fee calculations.
- 2.4.2 Man-Hours or Man-Days by Role and Phase:** The estimated effort required, typically provided by the user (project manager) based on scope complexity or historical data. The user will distribute the effort across project phases and team roles. *Format:* a matrix of hours (or days) with phases as columns and roles as rows, mandatory for fee calculation (the core driver of the Base Fee).
- 2.4.3 Travel Requirements:** Number of trips for project meetings or site visits, and number of people per trip. Either specified in RFP (some RFPs require a certain number of meetings) or estimated by the team. *Format:* integer counts (e.g. 3 trips, 2 people each), optional (defaults can be 0). Used to calculate reimbursable travel expenses.
- 2.4.4 Reimbursable Unit Costs:** Standard costs for travel and other reimbursables. For example, a fixed rate per person-trip for international travel (e.g. \$6,000 per trip) is defined as a constant in the database or configuration. Other reimbursables might include unit costs for printing, courier, etc., if applicable. *Format:* numeric costs, with currency; optional (required only if such expenses are anticipated).
- 2.4.5 Sub-consultant Fees:** If the project requires external consultants (e.g. local firms for surveys or specialty studies), their fees may be input if known (from their proposals or fee estimates). In many cases, the RFP might not provide this, so the user enters a provisional sum. *Format:* numeric amount per sub-consultant or total, optional (only if using sub-consultants; included in Financial Proposal breakdown as “Subconsultants”).

2.4.6 **Tax Rates:** Applicable tax or VAT percentage for the project jurisdiction (from internal knowledge base or user input). Many proposals present fees *exclusive* of taxes, but the rate is noted for contract purposes. Format: percentage, optional (used to calculate a gross fee if needed or mentioned in conditions).

2.4.7 **Currency and Exchange Rates:** The primary currency for the fee (e.g. JPY or USD) and any conversion rate if a secondary currency is shown. The system may use JPY internally (for rate consistency) and convert to USD or local currency for client presentation. Source: user input or financial database; Format: currency code and numeric rate, optional (mandatory if output requires multiple currencies).

## 2.5 Reference Projects Data:

2.5.1 **Project Profiles Library:** An internal database of the firm's past projects, particularly for Relevant Experience section. Each entry includes project name, location, scale (site area, project size), client name, project type/category, a brief description of the work, and optionally awards or outcome. Format: structured records (text and numeric fields, and images if available).

2.5.2 **Selected Relevant Projects:** The user (or automatically, based on project type and region) chooses a set of impressive past projects to include. Typically, 10-15 projects for each area of experience are included to demonstrate experience. Each chosen profile provides content for the Relevant Experience section. *Source:* user selection from database (the RFP itself might suggest areas of experience to highlight, but final choice is by the proposal team). *Format:* list of project IDs or names to include, mandatory (at least one reference must be included to complete that section).

Each input field above is marked as **mandatory** if the proposal generation cannot proceed meaningfully without it (e.g. Project Name, Scope Summary, man-hour estimates), or **optional** if the proposal can still be generated with defaults or omissions (e.g. Site Area, if not given, the proposal will omit that detail). The **source** indicates whether the data is expected to be parsed from the RFP text, fetched from an internal database, or provided manually by a user. In practice, after the RFP upload, the system will present the parsed results for user verification – the user can confirm or edit these fields before generation.

## 3 Output Document Sections and Content

Once the necessary inputs are prepared, the system generates the proposal document's sections. Each section of the output has defined content

with placeholders to be filled by the parsed or input data. Below, we define each section and explain how the system populates it:

### 3.1 Cover Letter

The Cover Letter is a formal letter addressed to the client, serving as the proposal's introduction. It appears on the proposal's cover or first page. The content is semi-static, following business etiquette, but includes dynamic placeholders for project-specific details. Key elements of the cover letter include:

- 3.1.1 **Address and Salutation:** Typically starts with *"Dear Sirs,"* or a specific addressee if known, followed by a brief introduction referencing the project name (e.g., *"...invited to submit our proposal for your esteemed project {{Project Name}}."*). The client name or organization may also be referenced here.
- 3.1.2 **Expression of Honor and Interest:** A sentence or two expressing gratitude for the opportunity and the firm's enthusiasm for the project.
- 3.1.3 **Brief Project Understanding Statement:** A custom sentence reflecting an understanding of the project's nature. For example, the cover letter might mention the project type and site, e.g. *"Based on our in-depth understanding of the project, we believe we can establish a solid master plan for the {{Site Area}} site in {{Location}} that offers an attractive environment..."*. This shows the client that the team is aware of the project specifics (the system can insert site area and location if available).
- 3.1.4 **Team Introduction:** The cover letter often highlights the strength of the proposed team. The system will insert a line about the multidisciplinary team assembled for the project, including key disciplines involved. For instance, an automatically generated line might read: *"We have assembled a highly skilled multidisciplinary team of experts in urban planning, architecture, landscape, and infrastructure under the leadership of our top management"*. This text can be drawn from a template but will adapt to the disciplines relevant to the project (if the project is, say, an architecture-only project, it might omit unrelated disciplines).
- 3.1.5 **Assurance of Commitment:** One or two sentences indicating senior oversight and commitment to deliver (often mentioning a senior executive or project director who will personally oversee the project). In the example content, the letter stated that the project will be overseen by a Senior Executive Officer, emphasizing a high level of commitment. The system can insert the name and title of the executive sponsor for the proposal (from a user profile or database).
- 3.1.6 **Closing and Signature:** A polite closing expressing hope to meet the client's expectations and a forward-looking statement (e.g. looking forward to the opportunity to work together). Then a sign-off with the name, title, and company of the signatory (typically a senior

officer). The system can fetch this from a configuration (e.g., the designated signatory's details).

- 3.1.7 *Population logic*: The cover letter content is primarily based on a **template** which can be stored in the database or as a text snippet in the system. Placeholders like **{{Project Name}}**, **{{Site Area}}**, **{{Location}}**, and references to disciplines or team leader name are replaced with actual data. Some sentences might be conditionally included (for example, mention of site area only if that data is available). The tone remains formal and sincere. The system may use an LLM to refine this section if needed – for instance, it could prompt the LLM with the project specifics and ask for a polite introductory letter. However, a standard template ensures consistency. The user can review and edit the auto-generated cover letter in the Proposal Editor UI before finalizing, to make any project-specific tweaks or ensure the tone fits the particular client.

## 3.2 Project Understanding

This section demonstrates the consultant's understanding of the project background, context, and the client's objectives. It is typically divided into one or two parts: a general **Site/Project Information** overview, and a **Client's Request or Objectives** part.

- 3.2.1 **Site/Project Information**: A narrative description of the project's context – often including the site characteristics, overall project goals, and key challenges or focuses. The system generates this by analyzing the RFP's background section. For example, it will mention the site size and location, and the major development themes. In a master plan project, the text might cover the intended development mix (residential, commercial, etc.) and key principles like sustainability or connectivity, as in this example: *"The project involves developing a comprehensive master plan for a vibrant and sustainable urban environment, integrating residential, office, and commercial spaces, and embodying principles of Transit-Oriented Development (TOD)."* The system can extract phrases directly from the RFP to ensure accuracy (e.g. if the RFP states specific goals, those will be reflected).
- 3.2.2 **Client's Request/Objectives**: This sub-section explicitly enumerates what the client is looking for, if stated in the RFP. It may reiterate the scope in terms of client goals or required outcomes. For instance, the proposal might state: *"The client requests a plan that incorporates TOD elements and caters to both current and future community needs, with a strong emphasis on sustainability and smart city methodologies, all within a tight timeline of **{{Project Timeline}}**".* Here, **{{Project Timeline}}** could be "11 weeks" or another duration extracted from the RFP. This part ensures the

client sees their key asks mirrored back, demonstrating understanding.

- 3.2.3 *Population logic*: The content for Project Understanding is generated by **summarizing the RFP**. The system's parsing module, often using a Large Language Model (LLM), identifies relevant portions of the RFP (such as project background, objectives, scope overview) and condenses them. The parsed information is mapped to structured data (e.g., project purpose, key features, timeline). An LLM can then be prompted to compose a concise narrative using those points. Alternatively, the system might use predefined template sentences into which it plugs details – for example, a template for site description and a template for client objectives. In either approach, placeholders like site area, number of hectares, specific development themes, or timeline are inserted from the parsed data. The output text is stored under a key (e.g., **{{Project Understanding}}**) and inserted into the document template. The user can edit this section in the Preview Editor if any nuance is missing or to adjust wording.

### 3.3 Methodology

The Methodology section outlines *how* the consultant team will execute the project. It typically breaks down the approach into phases or key steps and demonstrates a clear understanding of process and deliverables. The system uses a combination of standard methods and project-specific information to generate this.

A typical methodology for a planning/design project might include steps like: **Project Kick-off, Site Analysis, Workshops/Iterative Design Sessions, Concept Development, Refinement with Feedback, Final Proposal and Presentation**, and **Quality Control**. For example, a generated methodology could include:

- 3.3.1 **Project Kick-Off**: Organize an initial workshop with the client and stakeholders to confirm the project scope and strategy, and conduct a comprehensive site analysis.
- 3.3.2 **Strategic Workshops**: Conduct a series of design workshops to generate and evaluate master plan options, incorporating real-time feedback from the client. (If the RFP mentioned a certain number of workshops or interim presentations, those will be reflected here.)
- 3.3.3 **Conceptual Master Plan Development**: Develop multiple master plan options considering key factors such as urban density, transportation, land use, and sustainability. Integrate feedback and ensure compliance with local regulations at this stage.
- 3.3.4 **Client Involvement & Feedback**: After presenting initial options, hold follow-up sessions to refine the selected concept with client

input. Ensure the evolving design aligns with the client's vision and technical feasibility.

- 3.3.5 **Finalization & Presentation:** Finalize the preferred master plan incorporating all feedback and necessary analyses. Prepare a compelling presentation of the plan, including phasing, infrastructure integration, and expected outcomes.
- 3.3.6 **Quality Control & Delivery:** Conduct internal QA/QC to uphold quality standards and confirm that the final outputs meet all requirements and are ready for submission.
- 3.3.7 These steps are adapted to the project's specifics. For instance, if the RFP emphasizes environmental sustainability, the methodology will highlight tasks like environmental impact assessments or sustainable design reviews. If a strict timeline is given (e.g. 11 weeks), the methodology may mention fast-track or parallel work streams to meet the deadline.
- 3.3.8 *Population logic:* The system will likely have **methodology templates** for different project types. For example, a master plan project has a known sequence of activities (similar to the example above), whereas an architecture project might have a slightly different set (concept design, schematic design, design development, etc.). The appropriate template is chosen based on the **Project Type** input. This template is then adjusted: the number of workshops or phases might be updated to match the RFP (e.g., if the RFP explicitly asks for 2 workshops, the system ensures 2 are described). The LLM or rule-based text generator will insert project-specific details where relevant – for instance, naming the project in the objectives, or adding a note about local considerations (if the RFP mentions local codes or authorities, the methodology might include a step for regulatory coordination). All these steps are compiled into a formatted list or paragraph form. The content is stored under the **{{Methodology}}** placeholder for insertion. Users can edit or reorder steps in the editor if needed, but generally it should cover all standard steps to give the client confidence in the approach.

### 3.4 Scope and Deliverables

This section details *what* work will be done (scope) and *what* tangible outputs will be delivered (deliverables). It is typically presented as a bullet list or numbered list of items for clarity. The content here is drawn directly from the RFP's stated scope requirements and deliverables, ensuring the proposal aligns one-to-one with the client's expectations.

- 3.4.1 **Scope of Work:** The scope portion enumerates the tasks the consultant will perform. Each task can be a sentence or short paragraph. For example, tasks might include studies, analyses,

design tasks, and coordination efforts. If the RFP listed tasks (e.g., “1. Conduct site zoning analysis... 2. Develop functional land-use options... 3. Determine infrastructure requirements...”), the system will capture those. For instance, a scope item might read: “**1. Functional Cluster Zoning:** Analyze and propose functional zoning clusters for the site (e.g., residential, commercial, civic zones) considering utilities, road networks, and development restrictions.” Another might be, “**2. Development Phasing Strategy:** Identify and describe the main development phases, optimizing use of existing infrastructure and outlining necessary relocations or upgrades of facilities.” Each item corresponds to a requirement in the RFP or a logical task necessary for the project. The numbering and wording should mirror the RFP wording closely to show compliance – often even using the client’s own language where possible (the LLM can help paraphrase if needed for clarity, but important terms will be retained).

3.4.2 **Deliverables:** Following the scope tasks, a list of deliverables is presented. These are the outputs the client will receive at project completion or at phase milestones. Common deliverables include reports (e.g. Master Plan Report, Environmental Impact Report), drawings (plans, sections, maps), presentation materials, models, etc., depending on the project. The system extracts any explicit deliverable list from the RFP (for example, “*Deliverables: a) Interim report, b) Draft master plan, c) Final master plan document, d) 3D digital model, e) Presentation boards.*”). If the RFP isn’t explicit, the system can suggest typical deliverables for the given project type, which the user can then adjust. Each deliverable is listed with a brief description if needed. For example: “**Final Master Plan Report:** A comprehensive report including all analyses, plans, drawings, and recommendations for the master plan, in both hardcopy and PDF formats.” Such descriptions ensure the client knows exactly what will be handed over.

3.4.3 **Population logic:** The **LLM-based parser** plays a big role here. When the RFP is ingested, the prompt for the LLM includes instructions to extract the scope of work items and deliverables. The parsed output might be structured (e.g., a JSON with arrays for “tasks” and “deliverables”) which the system then formats as list items. The ordering and phrasing follow the RFP closely. The placeholders in the template (e.g., **{{Scope And Deliverables}}**) will be replaced by the formatted list of tasks and deliverables. If the RFP text for scope is very verbose (spanning multiple paragraphs or including sub-bullets), the system will condense it to a clear list but still ensure all main points are included. In cases where the RFP scope is vague, the proposal manager may manually enter or edit the scope list to ensure it’s precise. The final content is critical for

contractual clarity – it sets the boundaries of work – so the system ensures no item from the RFP is omitted. The user interface will allow reviewing this list before finalizing.

### 3.5 Schedule

The Schedule section provides an overview of the project timeline and key milestones. It aligns the proposed work plan with the client's timeframe. In a formatted proposal (especially in slides or Word format), this might be presented as a timeline graphic or table, but the textual description is as follows:

- 3.5.1 **Overall Duration:** The total project duration in weeks or months is stated upfront. For example, ***"Project Duration: Approximately 11 weeks from kick-off to final submission."*** If the RFP gave a specific deadline or duration, that is used; otherwise, the duration is estimated based on the scope (and can be adjusted by the user).
- 3.5.2 **Phase Breakdowns:** A breakdown of timeline by phase is provided, often in a table or list. Each project phase (as defined in the scope) will have a start and end or a duration. For example: ***"Stage 1 – Concept Development: 13 weeks"***, ***"Stage 2 – Detailed Master Plan: 9 weeks"***, etc., reflecting the effort allocated. In the case where multiple phases run sequentially, the timeline may be cumulative (Stage 1: Weeks 1–13, Stage 2: Weeks 14–22, etc.). If some phases overlap, that can be noted as well (though typically for simplicity, proposals assume sequential phases unless otherwise stated).
- 3.5.3 **Milestones and Meetings:** Key events are marked on the schedule. For instance, ***"Kick-off Workshop – Week 1"***, ***"Interim Presentation – Week 8"***, ***"Final Presentation – Week 11"***, etc. The example timeline in the Vietnam Master Plan included workshops labeled WS1, WS2, etc., aligned with stages. The system, using either a simple Gantt chart logic or milestone list, will populate these from the **Milestones/Workshops** input or derived from the methodology steps.
- 3.5.4 **Schedule Chart/Visual (if output format supports it):** In a PowerPoint or Word output, this section might include a visual timeline. In the FP-GEN template, for example, a slide showed a week-by-week timeline with markers for workshops. The system can generate this by populating a pre-designed chart or table with the phase names and durations. (For the purpose of this text brief, we focus on the data rather than the graphic.)
- 3.5.5 **Population logic:** The schedule is constructed by combining the **Project Phases** and the **Timeline** input. If the RFP specified an end date or total duration, the system uses that as the guiding parameter

and may distribute time across phases proportionally to their workload (or as indicated in RFP). If not specified, default durations for each phase can be suggested (e.g., Concept: 3 months, Detailed Design: 2 months, etc.) which the user can adjust. The system will ensure that the sum of phase durations equals the overall project duration. It will also insert any specific dates if provided (for instance, “Final submission due by **December 2025**” could be explicitly stated if known). In the template, placeholders like **{{Total Duration}}**, **{{Phase1 Duration}}**, **{{Phase2 Duration}}**, etc., or even an entire pre-formatted schedule table, are replaced with these values. The user can fine-tune the timeline via the UI, especially to adjust overlap or exact milestone weeks. The Schedule section thus communicates a clear plan that aligns with the client’s timeframe.

### 3.6 Team Structure

This section presents the proposed project team and organizational structure for the project. Its purpose is to reassure the client that the right expertise is on board and that roles and responsibilities are clearly defined. The output may be an org chart diagram or simply a list of team members by role, depending on the format. In FP-GEN’s slide format, a visual diagram was used, with placeholders for team roles and logos.

Key elements in Team Structure include:

**3.6.1 Key Personnel and Roles:** A listing of the main team members and their roles. This typically includes the Project Director/Principal-in-Charge, Project Manager, and Lead specialists for each major discipline involved (e.g. Lead Master Planner, Lead Architect, Lead Infrastructure Engineer, etc.). For each named individual, the title and department can be given. In a text format, this might be bullet points or a simple table (Name – Role). For example: “*Wataru Tanaka – Senior Executive Officer (Project Principal)*”, “*Jane Smith – Project Manager (Urban Planning)*”, “*John Doe – Lead Architect*”, etc. If the output is more visual (like a slide), these might appear in an organization chart layout with hierarchy (Project Principal at top, then project manager, then discipline leads below).

**3.6.2 Department/Discipline Grouping:** Often, team structure is presented by grouping personnel under their discipline or sub-team. For instance, under “**Master Planning – Overall Coordination**”, one might list the master planning team members; under “**Architecture**” list architectural team members, etc. The example text shows categories like *Masterplanning, Hotel, Residential* which likely correspond to sub-teams or specialist sectors for that project. Our system will generate these groupings based on the **Required Disciplines** and team input. If a project requires multiple specialized sub-teams (e.g., separate teams for different components like hotel planning, residential planning), those are

shown as branches in the structure.

**3.6.3 Specialty Consultants and Partners:** If external consultants are part of the team, they are included here as well. They might be shown separately (for example, alongside a “Specialist Consultant” label with their logo). In the template, there were placeholders for client and specialist consultant logos, indicating that, for instance, a local consultant’s logo could be placed to show partnership. In text form, we would list the consultant name and role (e.g., “ABC Environmental Ltd – Environmental Impact Assessment Consultant (Specialty Sub-consultant)”).

**3.6.4 Client/Stakeholder Representation (optional):** Sometimes proposals include the client’s team in the org chart for context. In FP-GEN’s template, a placeholder for **(CLIENT) Project Management** was present, suggesting the diagram showed how the consulting team interfaces with the client’s project manager. In our system, this could be included if the RFP mentions specific client liaison or if it’s customary to illustrate reporting lines.

**3.6.5 Population logic:** The Team Structure section is populated from the **Team Members input data**. The user will have entered or confirmed the list of team members and their roles. The system arranges this information according to a template. For example, the template might have an org chart shape with placeholders like **{{Project Director Name}}**, **{{Project Manager Name}}**, **{{Discipline1 Lead}}**, etc. Those get filled in by matching roles from the input. If an expected role isn’t filled (say no “Landscape Architect” is needed), that placeholder may be removed or left blank. If the output format allows images, the system can also insert profile photos or company logos where appropriate (e.g., the company logo for the prime consultant, client logo, and any partner logos). For a Word document output, a simpler representation (like a bullet list or table of team roles and names) is used, generated via a template.

Crucially, this section may not be heavily auto-generated by AI because it requires accuracy about names and positions – the data comes from the database of staff. However, the system could automatically fetch titles and department names for each person from an HR database. It ensures that all key roles requested by the client (if any were explicitly required in the RFP, such as “Key Expert CVs for Urban Planning and Transportation”) are accounted for in the team structure. The user can review this and adjust assignments before finalizing.

### **3.7 Financial Proposal**

The Financial Proposal section details the fee breakdown and payment

terms. It is one of the most critical sections, as it directly addresses the cost of the services. This section typically includes a breakdown of fees by phase or category, totals, and any clarifications on inclusions/exclusions, optional services, and taxes. The system generates this section through calculation algorithms and then formats the output (often as tables or structured text).

### Components of the Financial Proposal:

**3.7.1 Fee Breakdown Table:** A table or structured list summarizing the fee. This is usually broken down by project phase and/or cost category. For example, a table might have columns for each phase (Stage 1, Stage 2, ...) plus a Total column. Rows might include categories like “Nikken (Prime Consultant) Fees”, “Sub-consultants”, “Reimbursable Expenses”, etc., for each phase. Alternatively, the breakdown might list phases as rows with their fee, then additional rows for totals and optional items.

In our context, the system will likely present the fee breakdown by category first, with an overall summary, as in the example: **Nikken Subtotal, Overheads, Subconsultants, Production, Travel**, etc., culminating in the total fee. For instance, one generated breakdown might be:

| Category                      | Amount (JPY)       |
|-------------------------------|--------------------|
| Nikken Labor Fee (Base)       | ¥12,000,000        |
| Overheads & Profit            | ¥1,999,000         |
| Sub-consultants Fee           | ¥6,677,000         |
| Production (e.g. model, etc.) | ¥0                 |
| Travel Expenses               | ¥5,244,000         |
| <b>Total (Lump Sum Fee)</b>   | <b>¥25,921,000</b> |

(Figures above are for illustration; they match the example where the total fee is ¥25.91 million.) The system can also convert this to another currency if needed (e.g., ≈USD 235,552 in that example) and present both. If breaking down by phase, a separate table might list Phase 1 fee, Phase 2 fee, etc., and their sum equals the base fee.

**3.7.2 Calculation Algorithm (Fee Computation):** Behind the scenes, the system calculates these figures using input data:

**3.7.2.1 Man-hour Cost Calculation:** For each phase and each role, the system multiplies the **man-hours (or man-days)** by the respective **hourly (or daily) rate** from the rates database.

For example, if Phase 1 involves 20 hours of a Senior Planner (rate ¥16k/hour) and 10 hours of a CAD Technician (rate ¥5k/hour), it sums those costs. This is done for all roles in the phase to get the **Phase labor cost**. It repeats for each phase. Summing across phases gives the **Total direct labor cost** for the project.

3.7.2.2 **Overhead/Profit Addition:** Depending on the firm's policy, an overhead or profit margin is added. This could be a percentage of the direct labor cost. In the example, an overhead line item (~¥2 million) was about an 8.3% markup on the base labor subtotal. The system can either calculate this as a fixed percentage (configured in the database, e.g. 10% of labor) or use the difference between "staff-rate" and "billing rate" if the rates already include overhead. (If the internal database rates already have indirect cost separated, the system might choose to present the breakdown for transparency by splitting out that portion as "Overheads").

3.7.2.3 **Sub-consultant Fees:** If any sub-consultants are involved, their fees (input by the user) are added. They can be distributed by phase if applicable, or listed as a lump sum in one category. The example shows a "Subconsultants Subtotal" of ¥6,666,000 – presumably a known amount or percentage for that project. Our system will simply take the input values for each sub-consultant and sum them (if multiple), or allocate them to relevant phases if the user specifies.

3.7.2.4 **Reimbursable Expenses:** Travel and other reimbursables are calculated next. For travel, the formula is: **Number of Trips × People per Trip × Unit Cost per Trip**. For example, 3 trips × 2 people × \$6,000 = \$36,000, which would be converted to JPY (and in the example, travel came to ¥5,244k, likely similar logic). The system also includes any other reimbursables (like printing, communications) either as a flat percentage of labor or itemized if provided. In many proposals, travel is the main reimbursable item and is often a significant line.

3.7.2.5 **Contingency/Efficiency Adjustments:** Optionally, the system can include an efficiency or contingency factor. In the example sheet, an "Efficiency Target (%) = 54" was listed. This might indicate a planned efficiency saving (e.g., only charging 54% of something) or could be an internal metric. If the firm applies a discount or contingency, the algorithm would adjust the fee accordingly. For instance, if the target efficiency is 90%, the system might reduce the labor hours by 10% in the fee calculation to stay competitive, or explicitly show a discount line. This is configurable and not always present; mention it if the business uses such a factor.

- 3.7.2.6 **Total Fee Calculation:** The base labor cost, plus overhead/profit, plus sub-consultants, plus reimbursables yields the **Total Lump Sum Fee**. The system will sum these computed components to produce the total. It can present the total in multiple currencies as needed. In the example, the total was shown in thousands of JPY and also converted to USD. Our system can automatically convert using the provided exchange rate input, rounding as necessary for presentation.
- 3.7.2.7 **Optional Services:** The Financial Proposal should clearly delineate any optional items that are not included in the lump sum fee. Common optional services are **animations, physical scale models, special simulations or additional studies** beyond the base scope. The example conditions explicitly state that an animation can be provided at additional cost if requested, and that a physical model's costs (and import taxes) are excluded. In our proposal output, we will include an "Optional Services" note or sub-section. For instance: ***"Optional Service – 3D Animation: If the client requires a project animation video, it can be provided at an additional fee of USD X (to be determined based on scope)." Similarly, "Physical Model: A scale model can be produced upon request; costs for model fabrication and shipping will be quoted separately."*** These statements ensure the client knows these items are available but not in the base fee. The system may not automatically generate a price for these (unless the firm has standard rates for animations per minute, etc.), so it might simply flag them as available options. The text for these can be stored as boilerplate with slight modifications (like inserting the requesting party or project name if needed). The user can toggle inclusion of each optional item in the interface, and provide a price if they want to include a quote for it.
- 3.7.2.8 **Taxes and Terms:** Typically, the financial section will clarify tax matters and payment terms. The proposal should state whether taxes (VAT, GST, etc.) are included or excluded. The example states the fees are net receivable, excluding taxes and levies. Our system can include a standard line: ***"All fees are exclusive of any taxes or levies, which will be charged in addition as applicable."*** If a tax rate is known (say 10% VAT), it might say ***"(e.g., VAT at 10% may be added to invoices)."*** Payment terms are also included (e.g. ***"Payment shall be made within 10 days of invoice receipt"*** or a more standard 30 days, depending on company policy). These terms are usually static text or selected from a few standard options. The user can pick the appropriate terms template (some clients may have asked for a specific term in the RFP, which the parser might catch).
- 3.7.2.9 **Special Exclusions or Clarifications:** The financial

section often lists things not included in the fee to avoid scope creep. In the example, they explicitly exclude specialty consultant fees (like local consultants or surveys) from the lump sum, meaning the client would bear those separately or they'd be negotiated separately. Also, unusual costs like *"tax or charge for import of professional model"* are excluded. Our proposal will include similar statements as needed. The system has a set of standard **fee assumptions/conditions** that cover these points. For instance:

- 3.7.2.10 The fees are net of taxes (as mentioned above).
- 3.7.2.11 Payment to be made to a designated account within a certain period after invoicing.
- 3.7.2.12 Travel and accommodation are reimbursed at a fixed unit rate (specifying that rate, e.g. USD 6,000 per trip).
- 3.7.2.13 Specialty consultants (local consultants, surveys, authority approvals) are excluded from the fee and would be additional.
- 3.7.2.14 Costs for any physical models, special equipment, or their transport are excluded.
- 3.7.2.15 Optional services (like animation, additional presentations beyond scope) can be provided at extra cost.
- 3.7.2.16 These correspond almost exactly to the example's enumerated conditions, which our system can use as a template. The numbering and content of these conditions can be auto-generated unless the user customizes them. Typically, these are stored as boilerplate text keyed to project type or region (for example, the mention of Russia in the example conditions would be adjusted to the appropriate country or removed for a general case).
- 3.7.2.17 *Population logic:* The financial figures are computed by the **Fee Calculator module** of the system using the algorithms described. The results (all category totals, phase totals, etc.) are stored in structured form. The document template contains placeholders for these values or is designed such that the system can inject a table. For example, the template might have a placeholder **{{Financial Proposal}}** intended to be replaced by an embedded table or a formatted text block of the fee breakdown. The system could assemble a table in HTML or in the target format on the fly, or in the case of PPT generation, it might fill in text boxes or table cells with the computed values.
- 3.7.2.18 All the fee-related text (conditions, optional notes) are stored as standard text blocks in the database that can be concatenated under the **Financial Proposal** section. The user should review the calculated fees in the **Fee Calculator Interface** (described below) and can adjust inputs if something

looks off (e.g. they might tweak man-hours or add a missing cost) before finalizing. Once satisfied, the data flows into this section automatically.

- 3.7.2.19 Finally, the system ensures that the Financial Proposal section is **clear and transparent**: it presents the numbers cleanly and includes notes so the client understands exactly what is covered by the fee. This reduces back-and-forth and instills confidence that the proposal's cost is well thought out and justified.

### 3.8 Relevant Experience

The Relevant Experience section showcases past projects that are similar or otherwise demonstrate the firm's capability to successfully deliver the client's project. Each reference project is presented with key details and a brief description, often alongside images of the work. The system helps automate the selection and inclusion of these project profiles.

For each **reference project** included, the section will typically provide:

- 3.8.1 **Project Name and Type:** The title of the project and possibly the year or phase. For example, "*Ho Chi Minh City: Central District Urban Design Competition, Vietnam (Master Plan)*". The format might combine location and project type for quick relevance.
- 3.8.2 **Brief Narrative/Outline:** A short paragraph describing the project scope and significance. This is usually a few sentences highlighting what the project was about and any notable achievements (e.g., "won first prize in an international competition" or "completed on a fast-track schedule"). The narrative focuses on aspects relevant to the client's project. For instance, if the current proposal is for a master plan, the narrative will emphasize master planning experience. In the example text, the outline for Ho Chi Minh City described balancing heritage and redevelopment and noted that Nikken Sekkei won the first prize and was commissioned for the master plan. Such content is likely stored in the project database as the project's "abstract" or "description." The system will pull that text and possibly truncate or slightly tailor it to fit the space.
- 3.8.3 **Key Project Data:** A bullet list or inline mention of important metrics and facts about the project. Common data points include: **Location, Site Area/Project Scale, Client, Project Duration or Year, Awards** if any. These details give context and scale. For example: "*Location: Ho Chi Minh City, Vietnam; Site Area: 930 ha; Client: People's Committee of Ho Chi Minh City; Project Period: 07/2007 – 11/2007; Award: First Prize (international competition)*". The system retrieves these fields from the project

profile in the database. It formats them either as a list or as part of the narrative. In the example, they were listed after the narrative, which is a good approach to clearly present facts.

- 3.8.4 **Imagery (if applicable):** If the output medium supports it (e.g., in a Word or PPT with images), each project can include one or two thumbnail images (like a plan or completed project photo). The system would fetch the image file path from the database and embed it. In a text-only scenario, this may be omitted or replaced with an image caption. For this technical document, we focus on text, but it's worth noting image integration as part of the tool (with the UI likely allowing the user to choose which images to include for each reference). The example slides for relevant projects contained images with captions (such as "Bird's-eye View" or "Perspective of Riverfront Street with LRT" for the HCMC project); those were likely pre-prepared slides. Our system can simplify this by having a standard layout where text and images are inserted.
- 3.8.5 **Number of Projects:** The section typically has 2-4 projects. They may be grouped by category if needed (for instance, if the proposal covers multiple disciplines, one might show a planning project, an architecture project, etc.). Usually, for a master plan proposal, showing multiple master plan projects in similar contexts (e.g., other city master plans in Vietnam or the region) is most convincing. The system can suggest projects based on filters (same region, similar size/type) from the database. The user then confirms or modifies the selection.
- 3.8.6 *Population logic:* The Relevant Experience content is pulled from the **Project Profiles database** using the **Selected Projects** input. Each project's data is stored in a structured format, which includes the fields needed (name, location, scale, client, description, etc., and path to images). The system will have a template for presenting one project. For example, in a Word document, it might be a subsection or a table: *Project Name* (as a sub-subheading) followed by the description paragraph and then a bullet list of stats. In the FP-GEN slide template, they likely had dedicated slides for each reference project that could be imported or duplicated. In fact, the generator code hints at merging specialty slides from a "relevant" folder, suggesting each reference project might correspond to a slide that can be inserted if that project is chosen.
- 3.8.7 For implementation, we have two strategies:
  - 3.8.7.1 **Dynamic Template Filling:** Have a generic layout and populate it with each project's data. For example, iterate over selected projects and for each, replace placeholders in a copy of

a template segment. This approach is easier if exporting to a continuous document (like Word or PDF).

**3.8.7.2 Pre-made Slides or Sections:** Maintain a library of formatted slides/sections for certain well-documented projects (especially if each includes images and complex formatting), and insert those. This requires naming conventions and might not cover every new project, so it's more static. The code suggests some mechanism like this for slides.

**3.8.7.3** We will likely do the dynamic approach: The system will generate the text by inserting database fields into a pre-written narrative if available. Some projects might have long descriptions, so the system may truncate or summarize to fit a target length (the LLM could assist in summarizing a project description to, say, 100 words if needed). The key project data will be formatted uniformly.

**3.8.7.4** The user interface may allow the user to edit the description or highlight specific aspects for each project (for instance, to tailor a project description to emphasize a similarity with the current project). They can also reorder the projects' presentation. But by default, the system will produce a coherent Relevant Experience section demonstrating the firm's track record. For example, it might show two master plan projects in Vietnam and one in a similar context internationally, each with a success story or award mention to build credibility.

## **4 User Interface Design and Workflow**

The tool's user interface is designed as a step-by-step workflow that guides the user (typically a proposal manager or engineer) through inputting the RFP, verifying and editing extracted data, adjusting the fee calculation, and reviewing the final proposal before export. Below are descriptions of the four main screens (or stages) in the UI, along with their key elements, laid out as if they were wireframes:

### **4.1 RFP Upload and Parsing Screen**

**Purpose:** This is the starting point where the user provides the RFP document and initiates the automated parsing. It allows the user to see what data was extracted and correct or augment it.

**Layout and Elements:**

**4.1.1 File Upload Panel:** A prominent area or dialog to upload the RFP file (PDF/DOC). This includes a button "Upload RFP" which opens a file chooser, and once selected, a "Parse Document" button to start processing. The accepted formats are listed (PDF, Word, etc.).

**4.1.2 Parsing Progress Indicator:** After clicking parse, a progress bar or

spinner with messages like “Parsing RFP using AI...” is displayed, since this might take a bit of time (calling the LLM API).

4.1.3 **Extracted Data Form:** Once parsing is complete, the screen shows a form or a set of expandable sections containing the extracted fields. For example:

4.1.3.1 *Project Info:* shows text fields for “Project Name”, “Client Name”, “Project Location”, “Site Area”, etc., pre-filled with what the parser found. Each field is editable, so the user can correct any mistakes (e.g., if the RFP’s project title wasn’t clearly detected, the user can type it in). Mandatory fields might be highlighted with an asterisk.

4.1.3.2 *Scope & Deliverables:* shows a large text box or list for “Scope of Work Summary” (possibly the system’s summarized paragraph) and a multi-line field or list widget for “Deliverables”. The deliverables might be shown as a checklist or bullet list that the AI extracted. The user can edit text directly, add or remove items from the list.

4.1.3.3 *Timeline & Phases:* fields for total duration (if found) and a table or list of phases. For example, a small table with columns: Phase Name, Duration (weeks), Description – filled with data if parsed (or left blank for user to input if not in RFP). The user can add phases or adjust durations here.

4.1.3.4 *Team Requirements:* perhaps a summary of any specific team roles mentioned in the RFP (e.g., “The RFP requests an urban planner and traffic engineer”). This could be a notes field; otherwise, this might be blank if not parsed, and the user decides team later.

4.1.3.5 *Special Instructions:* any other relevant data the parser found (like “must include local consultant” or “specific budget limits”) shown as notes or additional fields.

4.1.3.6 *Parsed Text Viewer (optional):* A panel that shows the raw text of the RFP or the sections that the AI looked at, possibly side-by-side with the extracted data. This helps the user verify context. For example, if the AI pulled a certain sentence for scope, the user can see the original RFP excerpt to ensure it’s correct. This might be collapsible to reduce clutter.

4.1.3.7 *Alerts/Errors:* If the parser encountered any uncertainties or missing mandatory data, those are flagged. For instance, if the project name couldn’t be found, an alert might say “Project Name not detected – please enter it manually.” Similarly, if the PDF was scanned and text couldn’t be extracted properly, an error message would prompt the user to try another file or manual input.

4.1.3.8 *Navigation Buttons*: A “Next” or “Continue” button to move to the next screen (Proposal Preview) after reviewing the data. Possibly a “Save draft” button as well, to save progress. Also, a “Back” option if the workflow is revisited (though at the first step, back might just cancel).

4.1.4 Interaction: The user uploads the RFP, triggers parse. The system calls the parsing function which uses the ChatGPT API with a predefined prompt to extract all required info. When results come back, the UI populates the form fields. The user then goes through each field, verifying and editing as needed. They ensure mandatory fields are filled. Once satisfied, they click “Next” to proceed. Internally, the cleaned/confirmed data is saved (likely in a structured JSON or an object in the system, referred to as `parsedDataSet` in FP-GEN code). This dataset will be used to populate the proposal.

## 4.2 Proposal Preview and Section Editor

**Purpose:** In this stage, the user can preview the auto-generated proposal sections (narrative content) and make any edits or customizations to the text before finalizing. It’s essentially a live preview of the document sections (except perhaps the financial table, which might be shown in the next screen or here as a static snapshot).

### Layout and Elements:

4.2.1 **Section Navigation Menu:** On the left side or top, there could be a list of the proposal sections (Cover Letter, Project Understanding, Methodology, etc.). This could be a vertical list allowing the user to jump to any section by clicking it. The currently selected section is highlighted.

4.2.2 **Editable Preview Pane:** The main area displays the content of the currently selected section in a Word-processor-like view. Each section’s content is filled in with the generated text and placeholders resolved. For example, if the user clicks on “Cover Letter,” the pane shows the full cover letter text as it will appear in the proposal (address, body, closing). The user can click into this text and edit it directly if needed (e.g., tweak wording, add a sentence).

- i. For long sections like Methodology or Relevant Experience, the pane might allow scrolling. If the section content is very lengthy (exceeding some limit for one page), the system might show how it would be split (e.g., “page 1 of 2” for that section). In a slide-based preview, it might show one slide at a time – but since our output can be a Word doc, a continuous view is fine.

**4.2.3 Placeholders and Formatting:** The editor will ensure that any placeholders have been replaced with actual data. It could highlight any placeholder that remains unfilled (if for some reason something is missing, though ideally by this stage all placeholders are resolved). For example, if {{Team Structure}} was not filled because the user hasn't input team members yet, it would be highlighted for attention. However, team info comes later in the Fee Calculator step in our workflow, so more likely this preview at this stage might not include Team Structure or Financial content yet, or those sections might appear with a note "Will be generated after completing fee inputs".

- i. The editor supports basic text formatting (bold, italics, bullet lists). The methodology list or deliverables list should appear properly formatted as bullets or numbered lists. The user can edit text but likely not add completely new formatting structures beyond what's templated, to keep consistency.

**4.2.4 Pre-filled Content:** All sections from Cover Letter through Relevant Experience are shown. The Cover Letter, Project Understanding, Methodology, and Scope & Deliverables will be largely filled from parse results and templates as described. Team Structure and Financial Proposal might be empty or have placeholders if this editor is shown before the fee calculator step. It depends on whether the workflow allows going to preview before entering fee details. It might make sense that after parsing RFP, the next step is actually the Fee Calculator (to get all inputs done) then the preview. We can have some flexibility: either sequence works. Let's assume the user can go to the preview editor even before fee is set, but those sections will prompt them to fill in fee and team information.

**4.2.5 Guidance Text:** The UI might include a note at the top: "Review and edit the content of each section. You can modify wording, add details, or leave it as generated. Changes will be saved for the final output." This lets the user know they have control.

**4.2.6 Versioning/Reset Options:** Possibly an option like "Regenerate section" if they heavily edited an input and want the AI to regenerate that section's text. For example, if they change the Scope summary drastically, they might regenerate the Methodology to better align with the new scope. This could be a small "Regenerate" button next to the section title. Also, maybe "Restore default text" if they want to undo their manual edits and go back to the system's suggestion. (These are advanced features for convenience.)

**4.2.7 Navigation Buttons:** "Next: Fee Calculator" (if that step is

after) and “Back: RFP Data” to go to the previous screen. If the Fee step was already done, then “Next: Finalize & Export”.

4.2.8 **Interaction:** The user clicks through sections, reading the content. For example, in **Cover Letter**, they might see the salutation and decide to add the specific recipient’s name if known (“Dear Mr. Smith,” instead of “Dear Sirs,”). They simply type that change. The system autosaves or has a save button for that section. Next, they check **Project Understanding**; if the AI’s summary missed a point, they can insert an extra sentence or correct a fact. In **Methodology**, they might reorder points or add a detail (say the client requires a local approval process – they could add a bullet about “Local Authority Coordination” in methodology). The editor makes this easy with list editing tools. For **Scope & Deliverables**, they review the list to ensure it matches the RFP; they might add a deliverable that was not parsed initially.

4.2.9 If team and fee sections are not yet populated, the **Team Structure** section might show something like “(Team structure will be generated after defining team members in the Fee Calculator step)” with a link or prompt to go to that interface. Similarly, the **Financial Proposal** section might show a placeholder table or message “Fee breakdown to be calculated – go to Fee Calculator”. The user will then proceed to the Fee Calculator to input those details. In some workflows, it might make sense that after RFP parsing, the user goes to Fee Calculator before coming to this preview/editor for final text touches, especially since financial figures often inform narrative (for example, sometimes the cover letter might mention the fee or schedule in brief).

4.2.10 For this description, it’s fine either way. The key is the user has a chance to ensure all narrative text is polished. They do not have to worry about formatting in detail – the system maintains the style (fonts, heading sizes, etc.) as per the template, so the preview should reflect a near-final look.

#### 4.3 Fee Calculator Interface

**Purpose:** This is a dedicated interface for setting up and calculating the financial aspects of the proposal. It allows the user to input man-hours, assign roles, include reimbursables, and immediately see the fee breakdown and totals. It’s essentially a specialized form or spreadsheet within the tool.

##### Layout and Elements:

4.3.1. Phase and Role Matrix: The main element is a table where rows represent team roles (or individual staff) and columns represent project phases. The intersection cells are where the user inputs the amount of work (in hours or days) for that role in that phase. There may also be an additional column for “Total Hours” per role (summing

across phases) and a column for “Rate” and “Cost”. For example:

| Role/Staff               | Rate<br>(¥/hr) | Phase<br>(hrs) | 1 Phase<br>(hrs) | 2 Phase<br>(hrs) | 3 Total<br>Hrs | Cost (¥)        |
|--------------------------|----------------|----------------|------------------|------------------|----------------|-----------------|
| Urban Planner<br>(P-ST3) | ¥48k           | 100            | 40               | 20               | 160            | ¥7,680k         |
| Architect (A-<br>PR4)    | ¥98k           | 50             | 80               | 40               | 170            | ¥16,670k        |
| ... (other roles)        | ...            | ...            | ...              | ...              | ...            | ...             |
| <b>Subtotal</b>          |                |                |                  |                  | <b>330</b>     | <b>¥24,350k</b> |

4.3.2. The above is an illustrative partial table. The user can add or remove roles from this matrix. Perhaps there’s a dropdown of standard roles (populated from the rates database) to add a new row. When a role is added, its rate is automatically shown (from the DB). The user then fills in hours. The Cost cells (and subtotal) are auto-calculated by the system (hours \* rate). If rates differ by phase or if using specific staff names with unique rates, the table might have staff entries instead of generic roles – that depends on how granular the pricing is (usually role-based is enough). The table might be large, so it could be scrollable or split by phase if too many. Alternatively, some UI designs use one section per phase, listing roles and hours, which can be easier on smaller screens. But a matrix gives an overview of distribution of effort.

4.3.3. **Sub-consultants Input:** Below or alongside the matrix, there should be a section to input sub-consultant fees. This could be a simple list: For each sub-consultant, fields for **Name**, **Scope/Role**, **Fee Amount**. The user can click “Add Sub-consultant” to create a new entry (e.g., “Local Planner – ¥5,000k”). They can add as many as needed. These fees might not be distributed by phase (unless more detail is known). The UI just sums them up for inclusion in total.

4.3.4. **Reimbursables Input:** This section allows input of travel and other expenses. For travel, it might have fields like: **No. of Trips**, **People per Trip**, **Unit Cost per Trip**. If the unit cost is fixed by policy (like \$6,000) the field might be pre-filled or non-editable, except perhaps currency conversion if needed. As the user changes trips or people, an output field calculates “Total Travel Cost = (trips × people × unit cost)”. Other reimbursables might have a small list of common items (e.g., “Report printing – ¥X”, “Courier – ¥Y”) or just a general “Miscellaneous reimbursables – ¥Z”. The user can fill those if needed or leave at 0.

4.3.5. **Fee Summary Display:** As the user inputs data, a summary panel shows the computed totals in real time. This summary echoes what will appear in the proposal. It could look like:

- a. Nikken Labor Fee: ¥X (sum of all role costs)
- b. Overheads (10%): ¥Y (if overhead is automatically calculated, show it)
- c. Sub-consultants: ¥A (sum of sub-consultant fees)
- d. Reimbursables: ¥B (sum of travel + misc)
- e. **Total Fee:** ¥(X+Y+A+B)\*\*  
If a secondary currency is required, it might also show “Total Fee in USD: \$...”. The UI should also display any “Efficiency/Contingency” factors if they are applied – e.g., if the firm decides to only charge 95% of the computed fee as a competitive measure, there might be an “Efficiency factor” slider or input (like that 54% in the example, though 54% was unusual; more common might be, say, 90-100%). For simplicity, we might skip dynamic efficiency input unless it’s a known practice; otherwise, it can be a hidden advanced option.

4.3.6. **Optional Items:** Fields to specify optional service fees. Possibly a checklist: “Include 3D Animation (Y/N)” – if checked, allow entering a fee or note (or select from standard packages). Similarly “Physical Scale Model (Y/N)” and an input for cost or note if yes. These won’t add to the total (they’re separate), but the UI can note “Optional items are not included in Total”. This ensures the user defines them if they want to offer them. If not, the default is they remain “not included” and will just be mentioned as available in text.

4.3.7. **Assumptions/Conditions Text:** Perhaps a text box or list of standard fee conditions (like the 6 points about tax, payment, etc.) might be shown for review. The user can toggle or edit these. For instance, checkboxes next to: “Exclude taxes from fees”, “Payment 10 days after invoice” etc. If the project is in a different country, maybe the wording about tax could be changed (the example mentions Russia specifically, which might not apply elsewhere). The UI could allow editing the text of these conditions or choosing a region-specific template. It’s important these are correct, so giving the user control is good. If they don’t change anything, a default set (global default or region default) will be used.

4.3.8. **Calculation Engine:** As soon as the user inputs or changes a number (hours, fees, etc.), the interface recalculates the costs and updates the summary. This can be done with client-side scripting for responsiveness. There might also be a “Recalculate” button to force refresh if needed, but auto-calc on change is ideal.

- 4.3.9. **Navigation Buttons:** A “Done” or “Apply to Proposal” button to finalize the fee inputs and go back to the Proposal Preview (or directly to final export). Also a “Back to Editor” or “Previous” to return to the narrative editor if needed. Possibly also “Save” if they want to save the state and come back later.
- 4.3.10. **Interaction:** The user will use this like a spreadsheet for budgeting:
- a. They start by reviewing the roles listed. The system might pre-populate some roles based on the scope (for example, if the RFP is a master plan, it might pre-add roles like Urban Planner, Architect, Infrastructure Engineer, etc., guessing from the disciplines). The user can add any missing roles or remove irrelevant ones.
  - b. Then for each phase, they fill in the hours or days for each role. If they have an internal estimation already (maybe from prior similar jobs or an initial Excel), they’ll input those numbers. If not, they might iterate to see the fee outcome. The system helps by summing the hours and showing costs immediately.
  - c. The user notices the cost might be too high or low, and can adjust hours or consider adjusting overhead or efficiency. For example, if the total fee is overshooting the client’s budget (if known), they might reduce some hours or set an efficiency factor (like only 90% of hours billable).
  - d. They input any sub-consultant fees provided by partners or required by scope. For instance, the client might require a local traffic study, and the local firm quoted \$50k; the user enters that under sub-consultant.
  - e. They fill number of trips expected. Say the project might need 2 trips for 3 people each, they input that. The system multiplies by the \$6,000 rate (which is either fixed or adjustable) and shows e.g. \$36,000 (~¥ x million) in travel.
  - f. If the user wants to include an optional animation, they tick that and maybe input “\$15,000” as the estimated cost. This won’t add to total but will be recorded to mention later.
  - g. They double-check the summary numbers. If overhead is automatically 10%, they verify that line. If they need to tweak overhead (some proposals might list overhead as a fixed number or a different breakdown), they adjust accordingly (the UI might allow editing the overhead % or amount directly).
  - h. They review the conditions list and maybe deselect one if not applicable (e.g., if working domestically, maybe travel is not needed, they could remove that clause; or if the client is tax-exempt, they might modify the tax clause).
  - i. Once satisfied, they click “Done” or “Update Proposal”. The system then takes these values and integrates them into the proposal data structure. It would update the **Financial**

**Proposal** section's placeholder content (generating the final table or breakdown text, and ensuring the conditions/notes are included).

This interface ensures that the financial part is done with high transparency and control. It prevents calculation errors (since formulas are built-in) and allows quick scenario adjustments. Essentially, it replaces manual Excel calculations (like those in the provided spreadsheets) with an interactive tool. The result is that when the user goes back to the Proposal Preview, the **Team Structure** section can now be filled (since roles/hours imply which disciplines are involved and the user can assign names to those roles if not already) and the **Financial Proposal** section will show the completed table and fee notes exactly as computed.

#### 4.4 Final Export and Output

**Purpose:** The last step lets the user generate the final proposal document in the desired format (Word, PDF, PowerPoint, etc.), and provides a final review checkpoint.

**Layout and Elements:**

4.4.1. **Document Preview (thumbnail or outline):** On this screen, the system might show a miniature preview of the entire document – for example, thumbnails of each page or slide – to give the user one last overview. If it's a Word/PDF output, maybe a scrollable preview of the pages. If PowerPoint, thumbnails of slides. This preview is not primarily for editing, but to verify formatting (like page breaks, image placements, etc.).

4.4.2. **Export Format Options:** Options to choose the format of the output. Possibilities:

- a. "Export as Word Document (.docx)"
- b. "Export as PDF"
- c. "Export as PowerPoint Slides"  
There could be radio buttons or a dropdown to select one or multiple. (The prompt specifically says "formatted for inclusion in Word", so Word/PDF might be the main ones, but since the prototype was PPT-based, including that as an option might be useful for internal use).

4.4.3. **Template Style Selection (if applicable):** If the company has multiple proposal templates or themes, the user might select which template to apply. For example, a different template might rearrange the order of sections or have a different visual design. In many cases it's fixed, but it's a consideration (for instance, a simplified template for short proposals vs a comprehensive one). For now, assume one

standard template.

- 4.4.4. **File Naming:** A field or suggestion for the output file name, e.g. “Fee Proposal - [Project Name].docx”. The user can edit this if needed (to follow any internal naming convention or client reference number).
- 4.4.5. **Export Button:** A big button “Generate Document” or “Export Proposal”. Clicking this triggers the document assembly. The system will take the compiled content (now fully populated with all sections text and financial tables) and merge it into the chosen format using the template. In the case of Word/PDF, it might use a document generation library or service. In the case of PowerPoint, as FP-GEN did, it might automate PowerPoint or use a library to fill a PPT template. The user sees a loading indicator while the file is being generated and saved.
- 4.4.6. **Post-Export Confirmation:** Once done, a message “Your proposal has been generated successfully!” is shown with a link or button to download the file. Possibly also note where it’s saved in the system if stored on a server or cloud.
- 4.4.7. **Quality Checklist:** The screen might also list a few final checklist items as reminders, such as:
  - a. “Verify that all client names and references are correct.”
  - b. “Ensure no highlighted placeholders remain.”
  - c. “Proofread the edited text for any typos.”  
and maybe require the user to check them off (as a gentle QA step). This is optional, but can be helpful in practice.
- 4.4.8. **Return/Modify:** If the user spots something in the preview thumbnails that needs changing (e.g., a section is too long and spilling into an extra page awkwardly, or an image is cropped), they can click “Back to Edit” which might take them to the relevant section’s editor or to the fee calculator if the issue is financial. This final stage is intended as review, so going back is still allowed.
- 4.4.9. **Interaction:** On this screen, after choosing format and clicking “Generate,” the system will compile the data. Under the hood, as seen in FP-GEN, the PopulatePresentation or equivalent function will iterate through the template, replacing placeholders with actual text. If the content in a placeholder is too large for one slide/page, it may duplicate slides or adjust layout to fit. For Word output, it might just flow into multiple pages as needed. The user’s edits are already captured in the content data, so they appear in the output.
- 4.4.10. After generation, the user downloads and opens the file to ensure everything looks perfect. The expectation is minimal manual tweaking after export, but some might choose to fine-tune formatting in Word (especially if the template is generic). Because of this, providing

a Word format (which the user can manually adjust if needed) is useful – as requested.

- 4.4.11. At this point, the business development team can send the document to the client. The tool could also archive the proposal data for future reference, but that’s beyond the immediate scope.

## 5 Automation & Integration Logic

Behind the user-facing features, the system integrates several automated components to parse text, query databases, and populate templates dynamically. This section describes how these pieces work together, guiding developers in implementation and highlighting points for maintenance.

- 5.1. **RFP Parsing via LLM:** The system uses a Large Language Model (LLM) (e.g. OpenAI’s GPT) to analyze the RFP text and extract structured information. A carefully crafted prompt (e.g. in `Prompt_all-parse.txt`) guides the model to output all necessary fields in a machine-readable format. For instance, the prompt might say: *“Extract the following from the RFP: Project Name, Client, Location, Site Area, Duration, Scope summary, Deliverables list, etc., and provide them in JSON format.”* The code calls the API with the entire RFP text and this prompt. The response is then processed – either by directly interpreting a JSON or by using a function like `DynamicExtractItems` to map the response into the `parsedData` dictionary. This approach means that any improvements to extraction (new fields or better parsing prompts) can be achieved by updating the prompt file, without changing code. It’s important to handle errors: if the LLM fails or returns incomplete data, the system should catch that and either retry with a simplified prompt or inform the user to fill those fields manually. The parsing logic is modular – encapsulated in a `ParseRFPText` function – so it can be maintained or replaced with other parsing techniques if needed (for example, if a future specialized model for RFP parsing is available).

- 5.2. **Database Queries for Supporting Data:** The system relies on internal databases for supplementary information:

- a. **Rates Database:** A structured table of roles/ranks and billing rates is stored (this could be a simple table in a JSON, CSV, or actual database table). When the user adds a role in the fee calculator, the system looks up its rate. For example, if “Urban Planner (P-ST3)” is added, the system finds the corresponding rate (e.g., ¥59k/hour) and overhead portion (¥11k) as per the rates table. This database should be maintained by the finance team – if rates change annually, updating the table will automatically update calculations. The integration should be such that rates can be

fetches by a key (rank code or role name). For extension, it might even handle different rate sets for different regions or client types.

- b. **Project References Database:** A repository of past project info is connected for the Relevant Experience section. The system likely queries this by project name or filters (perhaps using the Project Type or Location of the current project to suggest similar ones). The integration could be via a simple search or a direct retrieval if the user picks from a list. The key fields (description, site area, etc.) are pulled and inserted into placeholders in a project description template. If images are stored, their file paths are retrieved for embedding. Maintenance of this database (adding new projects) will directly enhance the tool's library – new entries become available to showcase. It's important that the data is kept consistent (e.g., always have a short description, location, client, etc., for each) to avoid gaps in the output.
- c. **Templates and Snippets Storage:** The various boilerplate texts (cover letter template, methodology steps per project type, fee condition clauses, optional services descriptions) are stored in a manner that is easy to update. This could be a JSON or YAML file or even a small CMS. For example, fee condition clauses 1–6 might be stored as a list of strings so that if legal or management teams update the wording or want to add a new clause, they can do so without changing code. The system would then iterate through that list to produce the numbered conditions in the output. Similarly, methodology templates for “Master Plan” vs “Architectural Design” proposals can be stored separately and keyed by project type. The integration here is to have a configuration layer that the generation engine pulls from. This ensures that as the company's standards evolve, the tool can be maintained by updating templates and data rather than logic.

- 5.3. **Dynamic Placeholder Replacement:** The system uses a template-driven generation approach. The final document (whether Word or PPT) is generated from a template file (e.g., FeeProposalFormat.pptx in the prototype) which contains textual placeholders demarcated by a special syntax (e.g., {{PlaceholderName}}). During generation, each placeholder in the template is replaced with content from the parsedData or other computed data structures. For instance, {{Project Name}} is replaced by the actual project name, {{Scope And Deliverables}} is replaced by the assembled bullet list of scope tasks, and so on. The code iterates through each slide/page and each shape/text element to find these markers and substitute text. It even handles if content is too long: in the PPT generator, if a text exceeded the slide's capacity, it duplicated the slide to continue the text. In a Word document, it might just flow to next pages automatically, but we need to ensure no placeholders remain.

- 5.4. For development, this means that any new fields we decide to use must have corresponding placeholders in the template and keys in the data. It's straightforward to add new ones (just add a placeholder in the template and populate that key in code). Also, if some section is removed or renamed, both the template and code need update. The template file acts as a view layer: non-programmers (designers) can update the styling in the template (company logo, colors, cover page design) without altering the generation logic, as long as they don't remove placeholder tags.
- 5.5. **Inserting Pre-formatted Slides/Sections:** Not all content is pure text replacement. For example, the Relevant Experience projects might be better handled by duplicating a pre-formatted slide/page for each project (especially when images are involved). The FP-GEN code indicates a function to insert specialty slides. We interpret that as merging slides for either specialty consultants or relevant projects. The integration design here could be:
- a. Maintain a library of template slides for reference projects (with image placeholders, etc.). When the user picks projects, copy those slides in and then do text/image insertion on them.
  - b. Or use a generic "Relevant Project" slide template and programmatically fill and duplicate it for each project. Either way, the generator needs to dynamically manipulate the document structure (add pages as needed). In Word, this could be done by inserting sections for each project based on a template section. We ensure the automation supports that: e.g., a loop that goes through each selected project and injects content.
- 5.6. **LLM for Text Generation:** Besides parsing, we might use LLMs for generating some narrative content (especially Cover Letter and Project Understanding) to add polish. For example, after extraction, the system could send a second prompt like: *"Using the following data: [project info], [scope summary], draft a Cover Letter for a proposal."* This would let the AI produce a nicely flowing letter which we then lightly post-process and fill in placeholders (like ensuring the letter includes the assembled team info sentence). Similarly, the methodology could be refined by an LLM – e.g., if the standard template exists, an AI could adjust language slightly to avoid it being identical every time or to incorporate a particular project nuance. However, reliance on AI for generation should be controlled: to maintain consistency and avoid unapproved text, the system mostly uses templates and only uses AI to *supplement* (like paraphrasing or summarizing RFP content). For implementation, these generation calls can be optional or user-triggered ("Regenerate section with AI" button, as noted in the preview editor). Each call would use a prompt specific to that section and the data available. Developers should ensure the prompt instructs the model to stay factual (not introduce things not in RFP) and possibly to output in a format that can be easily parsed if needed (though

free text is fine if we directly inject it). Also, include a token limit and fallbacks, as these models have cost and reliability considerations.

- 5.7. **Maintaining Specialty Costs and Rules:** The logic for specialty items (travel rate, whether to include certain consultants or not) should be configurable. For example, the travel reimbursement rate of \$6,000 per trip might change over time or differ by region. Instead of hard-coding it, it could be a value in a settings file or database table (maybe tied to region or year). Similarly, rules like “exclude local consultant fees from base” or “tax excluded” might have exceptions in some contexts. It’s important that these can be toggled without code changes. Possibly a configuration panel for admin users could be provided to adjust these global settings. In integration terms, the system should reference these settings wherever relevant (e.g., when generating fee condition text or calculating travel).
- 5.8. **Quality Assurance and Edits Handling:** Automation is great, but the system should handle cases where the user makes manual changes that conflict with automated content. For example, if the user deletes a placeholder or a whole section in the editor, the system should not crash when generating the document (it should detect missing placeholders and perhaps warn the user or simply skip filling those). Also, if the user edits the financial figures in the preview text (which they shouldn’t, because the official way is via the calculator), the system should either prevent editing those fields or override them with the calculator values on final export to avoid inconsistency. Probably the best approach: make the financial table non-editable in the preview (or if it’s text, at least flag that it’s auto-calculated).
- 5.9. **Auditing and Version Control:** Every proposal generated could be saved (inputs + final output) to allow reference and to help improve the system. For development, logging each step’s data (like what the LLM returned, what the final values were) can help debug and refine the prompts and calculations. Also, keeping versioned templates (with placeholders) is important so that when the template is updated, it’s compatible with the code’s expectations. A small version number could be embedded (like the example Excel had v1.0 in it) to track that the data format/template version matches the code. This way, if format changes (say we add a new section), the system can detect version mismatch and guide updates.
- 5.10. By combining intelligent text parsing, database-backed data, and template-driven generation, the FP-GEN tool ensures a high degree of automation while still allowing human oversight and customization. Each integration point is designed for maintainability: update prompts to improve parsing, update templates to refresh the document design, update rates and text snippets to reflect current standards – all without altering the core code. The development team should focus on robust error handling (for example, API failures, missing data) and making the

interfaces intuitive, while the business team should keep the reference data and templates up-to-date. This collaboration will result in a tool that significantly speeds up proposal preparation and maintains consistency and accuracy across proposals.

## **6 Handling Specialty Consultants, Reimbursables, and Optional Items**

Special attention is given to how the system deals with elements that are often outside the core scope or fee but need to be clearly addressed in the proposal:

**6.1. Specialty Consultants Fees:** These are fees for third-party specialists (e.g., local consultants, surveyors, special studies) that the firm might coordinate but not perform themselves. The system's default stance (as per company practice) is to *exclude* these costs from the lump sum fee unless specifically asked to include them. In the proposal's Financial section, the conditions explicitly state that such specialty consultant fees are not included. Practically, this means in our fee calculator, any sub-consultant fees listed are added to the total fee *if the intent is to include them in the contract*. If instead the intent is to have the client pay those directly, we might list them as "Excluded – to be contracted separately by client" in the text. The tool should allow either approach. Likely, by default, sub-consultant fees entered are assumed to be included (we add them to total), but the condition says they're not included to clarify none have been added unless explicitly listed. To avoid confusion: if the user enters a sub-consultant fee in the calculator, that means we intend to include it and pass it on to client (maybe as part of our contract). If they leave it zero or blank, but a consultant is needed, they would mention in conditions that those will be extra. So, the UI might allow marking a sub-consultant as "not included in fee (client-hired)" vs "included in fee". Those marked not included would not sum into total and instead be footnoted. This nuance ensures the proposal is transparent. The system should generate phrasing accordingly, e.g., "Local consulting fees (e.g., geotechnical investigation) are excluded and assumed borne by Client." versus if included, they'd simply appear in the cost breakdown. Maintaining clarity here is crucial to avoid disputes later.

**6.2. Reimbursable Rates and Policies:** Reimbursables are expenses like travel, accommodation, printing, etc., incurred during project execution. The system automates travel cost calculation using a fixed rate per trip (which is a policy set by the firm). In the example, \$6,000 per person-trip was used. This figure should be configurable (some projects might use a different flat rate, or actual cost reimbursement). The system clearly states the reimbursement basis in the fee conditions: *"International travel and accommodation shall be reimbursed at a fixed rate of USD 6,000 per person-trip, based on actual number of staff traveling"*. Our tool will

include that exact language or a variation adapted to the project context. For instance, if the project is domestic, travel might be lower or stated differently. The key is that the system does not usually include travel in the lump sum fee – it is listed either as a separate line that might be estimated or left as “to be reimbursed as incurred.” In the breakdown earlier, travel was shown and added to total, implying they estimated it up front. Some clients prefer a capped estimate for travel in the proposal. Our tool can do both: if the user enters travel as a cost, it’s included in total; if they leave it zero, the condition would state travel will be billed as incurred. This can be a toggle “Include travel in Lump Sum or Not”. The default might be *included as a fixed amount for proposal purposes* but always clarified. Similarly, other reimbursables (like communication, deliveries) can be either included or excluded. Typically, firms include small reimbursables in overhead but exclude big ones like travel. So our default content will state travel separately, and perhaps note that normal office expenses are included. The maintenance of the \$6,000 rate and any other fixed rates (like per diem if any) is done in a config file, so updating it company-wide updates all new proposals.

**6.3. Animation/Model Costs:** These are treated as *optional add-ons*. The example’s condition 6 says animation can be provided at additional cost if requested. Our system will handle this by not including any animation cost in the base fee calculation unless the user explicitly adds an optional item. In the UI, if the user checks “Animation” and enters a cost, we do **not** add it to the total fee; instead, we prepare a note like *“Animation services can be provided for an additional fee of approximately USD X, if requested.”* in the proposal. If the user leaves it unchecked (the usual case if not asked by client), the proposal will simply include the generic statement that it’s available optionally (without a price, or maybe “cost to be negotiated”). The same goes for physical models: often, clients don’t request them in initial RFP, but the firm may mention availability. So the condition about the physical model being excluded and subject to separate charge covers that. If a specific model cost is known or likely (some clients ask for an optional line item), the user can add “Physical model: \$Y” as an optional cost. The proposal then would list it under a subsection “Optional Services (excluded from base fee):” listing each with its price or note. This approach ensures clarity: the client sees what extras they could ask for and knows those aren’t in the current price.

**6.4. Taxes:** While not a service, handling taxes is important in proposals. The tool will include by default a statement that taxes (e.g., VAT) are excluded. If the user knows the local tax rate, they might add “(+ VAT at 10% if applicable)” to that statement. We should ensure the system’s output does not inadvertently include tax in the totals unless specifically instructed. Typically, proposals in international contexts leave tax out (since the client will add VAT when paying). Our system will follow that norm and list fees as net amounts. If needed, a separate line could be

shown for “Tax (VAT 10%) = X” and a “Gross total = Y” for clarity, but only if required by client. Otherwise, a simple note suffices. This is configurable by region.

- 6.5. In summary, the system’s logic is to **segregate core fees vs additional costs** clearly. All these specialty and optional items are either kept out of the base fee or highlighted as separate. The automatically generated text (fee conditions and optional services notes) in the proposal ensures the client is aware of what is not included. This prevents misunderstandings like the client assuming an animation is included or that the fee is gross. The development team should implement these as template-driven content with fillable blanks for any figures. The business team should review these clauses periodically to ensure they match current policy and project practices.
- 

## 7 **Conclusion:**

The Fee Proposal Generation System, as outlined above, provides a detailed blueprint for building an application that significantly streamlines the creation of professional fee proposals. By covering all major sections of a proposal document and integrating intelligent parsing, database resources, and user-friendly interfaces for editing and calculation, the tool will enable both business development and technical teams to produce high-quality proposals more efficiently. The approach balances automation with user control, using FP-GEN’s proven logic and real project examples as a foundation. Maintaining this tool will involve updating data (rates, templates, content snippets) and continuously refining the AI parsing prompts to improve accuracy. With this technical brief as guidance, the development team can proceed with implementing the system, and the business team can prepare to populate it with the necessary data and templates. The end result will be a robust, maintainable solution that cuts proposal preparation time, ensures consistency, and allows the firm to respond swiftly and effectively to new RFP opportunities.